

Documento de Arquitetura de Software (DAS) - CoMangá

1. Introdução

O CoManga é uma plataforma para catalogação e gerenciamento de coleções físicas de mangás. O domínio organiza Obras, Edições brasileiras e Volumes físicos, permitindo que o acervo seja administrado e consultado em uma experiência pública de catálogo.

A implementação atual reúne autenticação, administração de usuários e opções de domínio, cadastro de Obras, Edições e Volumes, importação de capas, vitrine pública, detalhes públicos e listagem de Obras por Autor. Estante Digital, Lista de Desejos, calendário público e o enriquecimento autenticado das respostas públicas permanecem previstos para as próximas etapas.

Este documento registra as decisões arquiteturais efetivamente adotadas, com foco em integridade dos dados, segurança, privacidade, manutenção e evolução gradual do produto.

2. Representação Arquitetural

O CoManga utiliza arquitetura cliente-servidor em três camadas: Apresentação, Aplicação e Persistência. A stack principal é PostgreSQL, Express, React e Node.js, com TypeScript no frontend e no backend.

O fluxo principal é: navegador -> SPA React hospedada na Vercel -> API REST hospedada na Render -> PostgreSQL no Neon. O backend também integra SMTP para mensagens de conta e Cloudflare R2 para o armazenamento de capas processadas.

2.1. Camada de Apresentação (Frontend)

O frontend, no repositório comanga-web, é uma Single Page Application construída com React, TypeScript e Vite. React Router controla as rotas no navegador; Axios centraliza as chamadas HTTP; Tailwind CSS compõe a interface; Vitest e React Testing Library suportam os testes da camada visual.

A aplicação possui rotas públicas de pesquisa, Obras, Edições, Volumes e Autores, além de rotas de perfil e administração. O código é organizado por funcionalidades, com componentes compartilhados separados das páginas e serviços de cada

domínio.

O Axios usa `VITE_API_URL` ou `/api` como base e envia credenciais junto às requisições. Em produção, a Vercel redireciona `/api` para o backend na Render, preservando o uso do cookie de sessão.

2.2. Camada de Aplicação (Backend)

O backend, no repositório `comanga-api`, utiliza Node.js, Express e TypeScript e expõe uma API REST. Ele concentra regras de negócio, validação de entrada, autorização, visibilidade pública, tratamento de erros, logs com identificador de requisição e integração com os serviços externos.

A aplicação é publicada como um monólito modular na Render, e não como microserviços. Internamente, os módulos são separados por domínio: autenticação, usuários, administração, catálogo e catálogo público. A arquitetura se inspira em Clean Architecture e Ports and Adapters de forma pragmática; alguns módulos ainda acessam o Prisma diretamente.

O fluxo usual é rota Express -> middleware -> schema Zod -> controller ou módulo -> regra de negócio -> Prisma -> mapper ou DTO -> resposta JSON. As rotas são organizadas em `/api/auth`, `/api/users`, `/api/admin` e `/api/public`.

2.3. Camada de Persistência e Mídia

O PostgreSQL é hospedado no Neon e acessado exclusivamente pelo backend com Prisma ORM. O frontend nunca se conecta diretamente ao banco. Há bancos Neon separados para desenvolvimento, testes e deploy, evitando que operações de cada ambiente afetem os demais.

A modelagem central contempla Usuário e Sessão, além da hierarquia Obra -> Edição -> Volume. Relações com autores, papéis de autoria, gêneros, demografias, editoras, formatos e outras opções administrativas são modeladas de forma relacional. Chaves estrangeiras, constraints, índices e transações preservam a integridade necessária.

As mudanças físicas do banco são versionadas por migrations do Prisma. Uma migration já aplicada não é alterada; correções posteriores são registradas em nova migration para manter a reprodutibilidade entre os ambientes.

As capas não são armazenadas no PostgreSQL. O administrador informa uma URL de origem, o backend valida e baixa a imagem, gera variantes com Sharp, envia os arquivos ao Cloudflare R2 e persiste no Neon apenas os metadados e chaves internas. O domínio acessa esse recurso por meio do contrato `MediaStorage`, sem depender diretamente do SDK do provedor.

3. Segurança, Sessão e Visibilidade

O CoManga usa sessão stateful; não utiliza JWT. No login, o backend valida a senha com bcrypt, gera um token aleatório, armazena apenas seu hash SHA-256 na tabela `sessions` e entrega o token original em cookie `HttpOnly`. O token não é exposto ao JavaScript nem armazenado em `localStorage`.

Em rotas protegidas, o middleware lê o cookie, calcula o hash, localiza uma sessão válida e não revogada, confirma que o usuário está ativo e anexa as informações necessárias à requisição. O controle de papéis é aplicado no backend. O `ProtectedRoute` do frontend melhora a navegação, mas não substitui a proteção do servidor.

Rotas públicas aceitam sessão opcional. Sem sessão, ou com cookie inválido, a requisição é tratada como visitante. Em qualquer situação, Obras e Edições privadas nunca são retornadas. Conteúdo adulto é omitido para visitantes e também para usuários cuja preferência +18 esteja desativada.

- Cookies `HttpOnly` para reduzir exposição a XSS;
- Sessão persistida e revogável no banco de dados;
- Hash de senha com bcrypt e tokens de ativação com expiração;
- RBAC e validação de permissões no backend;
- Validação de entrada com Zod, variáveis de ambiente para segredos e CORS restrito às origens autorizadas;
- Regras de privacidade e conteúdo adulto aplicadas pelo backend, independentemente da interface.

4. Qualidade, Testes e Integração Contínua

O desenvolvimento segue TDD quando aplicável e mantém testes automatizados para regras de negócio, contratos de infraestrutura, rotas e componentes. No backend, Jest e Supertest verificam unidades e integrações da API. No frontend, Vitest e React Testing Library verificam componentes, páginas, contexto de autenticação e interações relevantes.

ESLint é utilizado nas duas aplicações. A integração contínua no GitHub Actions executa instalação de dependências, geração do Prisma Client, migrations no banco de teste, build TypeScript, testes automatizados, cobertura quando configurada e lint. A conclusão de um cartão inclui os critérios Gherkin, estados de carregamento, erro e vazio, responsividade e integração real entre cliente, API e banco de desenvolvimento.

5. Ambientes e Deploy

O projeto mantém ambientes separados para desenvolvimento, testes automatizados e deploy. Cada ambiente utiliza sua própria base Neon e variáveis de ambiente próprias. Migrations e ações de manutenção devem ser direcionadas conscientemente ao banco correspondente.

No deploy, o frontend é hospedado na Vercel, o backend é hospedado na Render, o PostgreSQL permanece no Neon, as capas processadas ficam no Cloudflare R2 e as mensagens de conta usam SMTP por Nodemailer. As plataformas realizam deploy a partir das branches configuradas, enquanto os segredos permanecem somente nas configurações de ambiente.

O backend gera o Prisma Client, aplica as migrations destinadas ao ambiente e compila TypeScript antes de iniciar o processo Node.js. O frontend é gerado pelo Vite e servido pela infraestrutura CDN da Vercel.

6. Metas e Restrições Arquiteturais

Custo controlado no MVP: a solução utiliza as camadas gratuitas ou de baixo custo de Vercel, Render, Neon e Cloudflare R2, observando os limites de cada provedor.

Integridade e privacidade: operações críticas preservam relações e consistência por meio de constraints, índices, migrations e transações. A API é a autoridade sobre sessão, permissões, visibilidade e conteúdo adulto.

Manutenibilidade e evolução: TypeScript, módulos por domínio, contratos de infraestrutura, Prisma, testes e separação de ambientes reduzem o risco de regressão. A arquitetura suporta a evolução planejada do calendário público, Estante Digital, Lista de Desejos e enriquecimento das respostas públicas sem exigir uma reestruturação completa.